



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 10

NOMBRE COMPLETO: EMILIANO GARCIA OLVERA

N° de Cuenta: 314256009

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 06

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 14/Mayo /2026

CALIFICACIÓN: _____

Objetivo de la practica

La practica tiene como objetivo animar mediante keyframes los engranes y los elementos moviles del objeto previamente cargado (modelo gun con sus dos gun_gear). La animacion debe ser controlada por teclado y se otorga un punto extra si los cuadros (keyframes) se escriben a un archivo .txt externo y se leen desde el mismo. Adicionalmente el modelo animado debe quedar incorporado al proyecto final.

Desarrollo y codigo

El punto de partida fue el codigo del previo 10 (donde el modelo gun ya estaba instanciado con sus dos engranes gun_gear y se controlaban por teclas Z/X y C/V de forma manual) y el ejercicio en clase E10 (donde se implemento el sistema de keyframes completo con interpolacion lineal entre cuadros). La idea fue unir ambos: tomar el sistema de keyframes del ejercicio y aplicarlo al modelo del previo, de modo que tanto el objeto como sus engranes se animen automaticamente.

Estructura del Keyframe

Para que un solo cuadro de animacion pueda llevar tanto el movimiento del objeto como la rotacion de los engranes, se amplio la estructura FRAME del ejercicio agregando los campos correspondientes:

```
typedef struct _frame
{
    float posObj_x, posObj_y, posObj_z;
    float giroObj;
    float rotEngrane;
    float posObj_xInc, posObj_yInc, posObj_zInc;
    float giroObjInc, rotEngraneInc;
} FRAME;
FRAME KeyFrame[MAX_FRAMES];
```

Los campos terminados en Inc son los incrementos calculados por la funcion interpolation() para pasar de un cuadro al siguiente en i_max_steps pasos.

Interpolacion lineal

La funcion interpolation() calcula los incrementos por paso entre el keyframe actual y el siguiente:

```
void interpolation(void)
{
    KeyFrame[playIndex].posObj_xInc =
        (KeyFrame[playIndex+1].posObj_x - KeyFrame[playIndex].posObj_x) / i_max_steps;
    KeyFrame[playIndex].posObj_yInc =
        (KeyFrame[playIndex+1].posObj_y - KeyFrame[playIndex].posObj_y) / i_max_steps;
    KeyFrame[playIndex].posObj_zInc =
        (KeyFrame[playIndex+1].posObj_z - KeyFrame[playIndex].posObj_z) / i_max_steps;
    KeyFrame[playIndex].giroObjInc =
```

```

        (KeyFrame[playIndex+1].giroObj - KeyFrame[playIndex].giroObj) / i_max_steps;
    KeyFrame[playIndex].rotEngraneInc =
        (KeyFrame[playIndex+1].rotEngrane - KeyFrame[playIndex].rotEngrane) / i_max_steps;
}

```

Funcion animate()

Llamada cada ciclo de reloj, avanza un paso de la interpolacion. Cuando se completan los pasos del cuadro actual se pasa al siguiente y cuando se acaban todos los cuadros la animacion se detiene:

```

void animate(void) {
    if (play) {
        if (i_curr_steps >= i_max_steps) {
            playIndex++;
            if (playIndex > FrameIndex - 2) { playIndex = 0; play = false; }
            else { i_curr_steps = 0; interpolation(); }
        } else {
            posObj_x   += KeyFrame[playIndex].posObj_xInc;
            posObj_y   += KeyFrame[playIndex].posObj_yInc;
            posObj_z   += KeyFrame[playIndex].posObj_zInc;
            giroObj    += KeyFrame[playIndex].giroObjInc;
            rotEngrane += KeyFrame[playIndex].rotEngraneInc;
            i_curr_steps++;
        }
    }
}

```

Renderizado del objeto y los engranes con jerarquia

Para que los engranes hereden la posicion y orientacion del gun se aprovecha una matriz auxiliar (modelaux). Asi el modelo padre se dibuja una vez con su transformacion completa y los engranes se reposicionan sobre el mismo padre, agregando su propio giro animado:

```

// gun (objeto)
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(posObj_x, posObj_y, posObj_z));
model = glm::rotate(model, giroObj * toRadians, glm::vec3(0,1,0));
modelaux = model;
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
gun_model.RenderModel();

// engrane izquierdo
model = modelaux;
model = glm::translate(model, glm::vec3(-0.3f, 0.25f, 0.6f));
model = glm::rotate(model,
    (rotEngrane + mainWindow.get_engranemovadicionalX()) * toRadians,
    glm::vec3(1,0,0));
gun_gear_model.RenderModel();

// engrane derecho (sentido contrario)
model = modelaux;
model = glm::translate(model, glm::vec3(0.3f, 0.25f, 0.6f));

```

```

model = glm::rotate(model,
    (rotEngrane + mainWindow.get_engranemovadicionalX()) * toRadians,
    glm::vec3(-1,0,0));
gun_gear_model.RenderModel();

```

La variable rotEngrane proviene del keyframe interpolado, y se suma el valor de get_engranemovadicionalX() del Window para conservar el control manual con las teclas Z y X del previo. De esta forma podemos ajustar manualmente la rotacion antes de guardar un nuevo frame con L.

Controles por teclado

Se conservo el esquema del ejercicio en clase ampliandolo para incluir el eje Z y la rotacion del engrane. Las teclas configuradas dentro de inputKeyframes() son:

ESPACIO: reproducir animacion | 0: habilitar reproducir de nuevo

1: habilitar movimiento

2/3: mover objeto en X+/X- 4/5: mover en Y+/Y- 8/9: mover en Z+/Z-

6/7: girar objeto +/- 90 grados Z/X: rotar engrane -/+

L: guardar frame P: habilitar guardar otro

O: escribir cuadros a keyframes.txt R: leer cuadros desde keyframes.txt

Lectura y escritura de keyframes a archivo (.txt)

Para acceder al punto extra que ofrece la practica se implementaron las funciones saveFramesToFile() y loadFramesFromFile() que guardan/leen los cuadros clave en un archivo de texto plano. La primera linea contiene el numero total de frames y cada linea posterior contiene los cinco valores de cada cuadro (x, y, z, giro, rotEngrane).

```

void saveFramesToFile(void) {
    FILE* f; fopen_s(&f, "keyframes.txt", "w");
    fprintf(f, "%d\n", FrameIndex);
    for (int i = 0; i < FrameIndex; i++)
        fprintf(f, "%f %f %f %f %f\n",
            KeyFrame[i].posObj_x, KeyFrame[i].posObj_y, KeyFrame[i].posObj_z,
            KeyFrame[i].giroObj, KeyFrame[i].rotEngrane);
    fclose(f);
}

void loadFramesFromFile(void) {
    FILE* f; fopen_s(&f, "keyframes.txt", "r");
    fscanf_s(f, "%d", &FrameIndex);
    for (int i = 0; i < FrameIndex; i++)
        fscanf_s(f, "%f %f %f %f %f",
            &KeyFrame[i].posObj_x, &KeyFrame[i].posObj_y, &KeyFrame[i].posObj_z,

```

```

        &KeyFrame[i].giroObj, &KeyFrame[i].rotEngrane);
    fclose(f);
}

```

Con la tecla O se escriben todos los cuadros que se han ido guardando con L hacia el archivo keyframes.txt y con la tecla R se cargan al iniciar la aplicacion para reproducirlos directamente con la barra espaciadora.

Cuadros por defecto

Para mostrar el resultado sin tener que generar los cuadros a mano cada vez, en el main se definieron 9 keyframes iniciales que combinan movimiento del objeto en X/Y/Z, giros de 90 y 180 grados y la rotacion continua de los engranes hasta varias vueltas. Esto se ve al presionar ESPACIO al iniciar el programa.

```

KeyFrame[0]={0.0f,0.0f, 0.0f, 0.0f, 0.0f};
KeyFrame[1]={0.0f,0.0f,-1.0f, 0.0f, 360.0f};
KeyFrame[2]={0.0f,0.5f,-2.0f, 0.0f, 720.0f};
KeyFrame[3]={0.0f,0.5f,-2.0f, 90.0f, 900.0f};
KeyFrame[4]={1.0f,0.5f,-2.0f, 180.0f, 1080.0f};
KeyFrame[5]={1.0f,0.0f,-1.0f, 180.0f, 1440.0f};
KeyFrame[6]={1.0f,0.0f, 0.0f, 180.0f, 1800.0f};
KeyFrame[7]={1.0f,0.0f, 0.0f, 360.0f, 2160.0f};
KeyFrame[8]={0.0f,0.0f, 0.0f, 360.0f, 2520.0f};
FrameIndex = 9;

```

Evidencias

Como evidencia adicional, al ejecutar el programa con la tecla O se genera el archivo keyframes.txt y con R se vuelve a leer.

Se incluyen capturas de pantalla de la consola mostrando los mensajes impresos al guardar frames con L, al escribir el archivo con O y al cargarlo con R, asi como el video de la reproduccion de la animacion al presionar la barra espaciadora.

```
Select F:\Fac\8voSemestre\Lab Grafica\Practica10\Practica\64\Debug\Proyecto.exe
=====
Practica 10: Animacion por Keyframes
=====
Teclas:
  ESPACIO : reproducir animacion
  0       : habilitar reproduccion de nuevo
  1       : habilitar movimiento (presionar antes de mover)
  2/3     : mover objeto X+ / X-
  4/5     : mover objeto Y+ / Y-
  8/9     : mover objeto Z+ / Z-
  6/7     : girar objeto +90 / -90 grados
  Z/X     : rotar engranes +/- (keyframe)
  L       : guardar frame | P: habilitar guardar otro
  O       : escribir frames a keyframes.txt
  R       : leer frames de keyframes.txt
=====

Mueve: 2/3(X) 4/5(Y) 8/9(Z) 6/7(giro) Z/X(engrane)
rotEngrane=-30.0
Frame 9 guardado | x=0.00 y=0.00 z=0.00 giro=0.0 engrane=-30.0 | (P para guardar otro)
Listo: presiona L para guardar otro frame
Frame 10 guardado | x=0.00 y=0.00 z=0.00 giro=0.0 engrane=-30.0 | (P para guardar otro)
Listo: presiona L para guardar otro frame
Frame 11 guardado | x=0.00 y=0.00 z=0.00 giro=0.0 engrane=-30.0 | (P para guardar otro)
Listo: presiona L para guardar otro frame
Frame 12 guardado | x=0.00 y=0.00 z=0.00 giro=0.0 engrane=-30.0 | (P para guardar otro)
Mueve: 2/3(X) 4/5(Y) 8/9(Z) 6/7(giro) Z/X(engrane)
posObj_x=0.50
Mueve: 2/3(X) 4/5(Y) 8/9(Z) 6/7(giro) Z/X(engrane)
posObj_x=0.00
Mueve: 2/3(X) 4/5(Y) 8/9(Z) 6/7(giro) Z/X(engrane)
posObj_y=0.50
Mueve: 2/3(X) 4/5(Y) 8/9(Z) 6/7(giro) Z/X(engrane)
posObj_y=0.00
Mueve: 2/3(X) 4/5(Y) 8/9(Z) 6/7(giro) Z/X(engrane)
giroObj=90.0
Mueve: 2/3(X) 4/5(Y) 8/9(Z) 6/7(giro) Z/X(engrane)
giroObj=0.0
Listo: presiona L para guardar otro frame
Frame 13 guardado | x=0.00 y=0.00 z=0.00 giro=0.0 engrane=-30.0 | (P para guardar otro)
Listo: presiona L para guardar otro frame
Frames guardados (14) en keyframes.txt
Listo: presiona L para guardar otro frame
Frame 14 guardado | x=0.00 y=0.00 z=0.00 giro=0.0 engrane=-30.0 | (P para guardar otro)
Listo: presiona L para guardar otro frame
Frame 15 guardado | x=0.00 y=0.00 z=0.00 giro=0.0 engrane=-30.0 | (P para guardar otro)
Frames cargados (14) desde keyframes.txt
-
```



Conclusiones

La animación por keyframes resulta mucho más flexible que la animación por banderas y condicionales: una vez implementado el sistema basta con definir cuadros clave y la interpolación lineal entre ellos se encarga de generar el movimiento intermedio. Al ampliar la estructura FRAME con los campos para engrane y posición en Z fue directo combinar el sistema del ejercicio con el modelo del previo y obtener una animación donde tanto el objeto como sus engranes se mueven de forma sincronizada. Guardar los cuadros a archivo permite además reusarlos en futuras ejecuciones, lo cual será útil al integrar la animación al proyecto final.